LearnTrack — Solution Approach Document

NavGurukul Hackathon 2026 | Challenge 4: Progressive Student Dashboard

Author: [Your Name] | **Role:** Full-Stack Developer

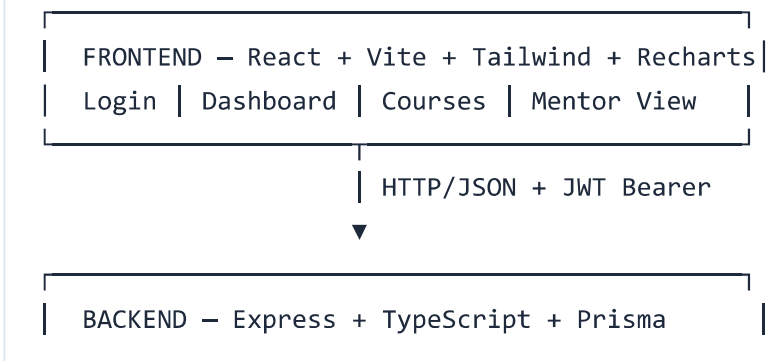
Repository: [Your GitHub URL] | **Date:** August 22, 2026

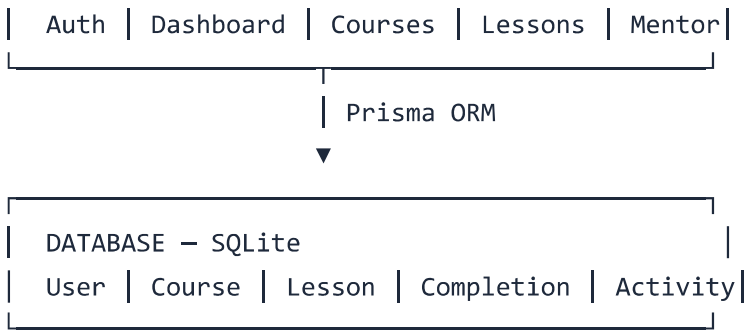
1. Problem Statement

Students need visibility into their learning progress — completed lessons, time invested, course-level breakdown, and guidance on what to study next. Mentors need a cohort-level view without manual tracking.

Requirement	Our Implementation
Email auth + roles	JWT + bcrypt; Student & Mentor roles
Progress dashboard	Summary API with stat cards & progress bars
Trend chart	30-day activity time series (Recharts area chart)
Pie/donut charts	Completion by course + overall status
Backend API	REST endpoints for auth, aggregates, lessons, activity
Seed data	Prisma seed: 3 courses, 18 lessons, 30 days activity
Stretch goals	Recommendations, CSV export, mentor dashboard, responsive UI

2. System Architecture

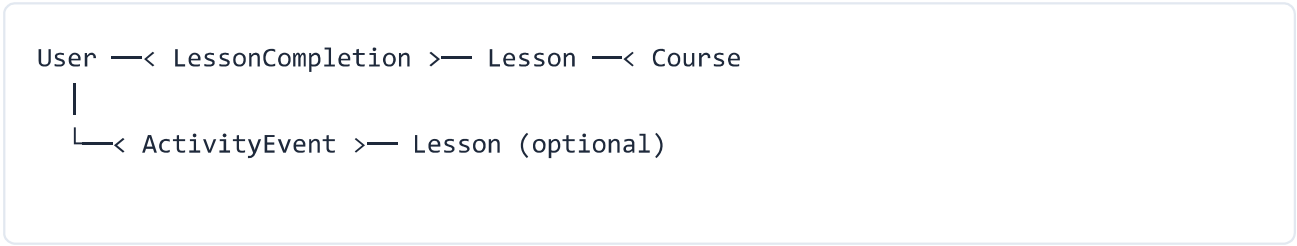




Technology Stack

Layer	Tech	Why
Frontend	React 18, Vite, TypeScript, Tailwind	Fast dev, component UI, responsive design
Charts	Recharts	React-native; area + pie charts easily
Backend	Express + TypeScript	Lightweight REST; type-safe server code
Database	SQLite via Prisma	Zero setup; perfect for local demo
Auth	JWT + bcrypt	Stateless; no session store needed

3. Data Model



Model	Purpose	Key Fields
User	Auth accounts	email, password (hash), role (STUDENT/MENTOR)
Course	Learning program	title, description, category
Lesson	Learning unit	title, durationMin, order, courseId
LessonCompletion	Completion milestone	userId, lessonId, completedAt (unique per user+lesson)
ActivityEvent	Time tracking	userId, minutes, date, type (LESSON/PRACTICE/REVIEW)

> **Key design decision:** Completions (binary milestones) are separated from ActivityEvents (continuous time tracking). A student can spend time on a lesson without completing it.

4. API Design

Endpoint	Purpose
POST /api/auth/login	Authenticate & receive JWT
GET /api/dashboard/summary	Stats, course progress, recent completions
GET /api/dashboard/timeseries	Daily minutes for trend chart
GET /api/dashboard/distribution	Pie chart data by course & status
GET /api/dashboard/recommendations	Next uncompleted lesson per course
GET /api/courses, /api/courses/:id	Course list & detail with progress

POST /api/lessons/:id/complete	Mark lesson done + log activity
GET /api/mentor/students	All students' progress (mentor only)
GET /api/export/progress	CSV download

Recommendation Algorithm

Rule-based (no ML): For each course, find the first lesson (by order) not yet completed. Return up to 3 suggestions with contextual reason strings. Simple, deterministic, and explainable.

5. Why This Solution Works

- **Complete MVP in scope** — All core + stretch features implemented and demo-ready in ~5 minutes
- **2-minute setup** — `npm install` + `npm run db:setup`; no Docker or external services
- **Clean architecture** — Frontend (UI) → Backend (logic) → Database (truth); easy to explain to judges
- **Extensible schema** — Swap SQLite → PostgreSQL; add enrollments, quizzes, ML recommendations later
- **100% open source** — No API keys or paid services; runs fully offline
- **Role-based access** — Students and mentors see different views from the same codebase

6. Drawbacks & Limitations

Limitation	Impact	Future Fix
SQLite	Poor concurrent writes at scale	Migrate to PostgreSQL
JWT in localStorage	XSS vulnerability	HTTP-only cookies + refresh tokens
Rule-based recommendations	Not truly adaptive	ML scoring by performance & spaced repetition
No real-time sync	Mentor must refresh to see updates	WebSockets or SSE
Simulated time tracking	Seed data is random; complete logs expected duration	Session timer or video watch-time API
No automated tests	Regression risk	Vitest unit tests + Playwright E2E
Single-tenant	No cohorts or org structure	Add Enrollment & Cohort models

7. Demo Instructions

```
# Backend:  cd backend && npm install && npm run db:setup && npm run dev
# Frontend: cd frontend && npm install && npm run dev
# Open:     http://localhost:5173

Student:  student@demo.com  / password123
Mentor:   mentor@demo.com   / password123
```

Demo flow: Login as student → dashboard stats & charts → open course → mark lesson complete → export CSV → logout → login as mentor → student overview.

8. Conclusion

LearnTrack prioritizes **clarity over complexity** — a working, locally runnable dashboard that maps directly to Challenge 4 requirements. The trade-off is production readiness vs. hackathon delivery speed, which is the right balance for a timed proof-of-concept with a clear path to production.
